



Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Алгоритми та методи обчислень

Лабораторна робота №1

«Поняття алгоритму. Задавання алгоритмів у вигляді блок-схем»

Виконав: студент 1 курсу ФІОТ, гр. ІО-41

Давидчук А. М.

Залікова книжка № 4106

Перевірив: *Порєв В. М.*

Тема: «Поняття алгоритму. Задавання алгоритмів у вигляді блок-схем» .

Мета: Навчитися створювати блок-схеми лінійного алгоритму; розгалуженого алгоритму та циклічного алгоритму за допомогою редактора блок-схем afse або іншого довільного редактора.

Хід роботи

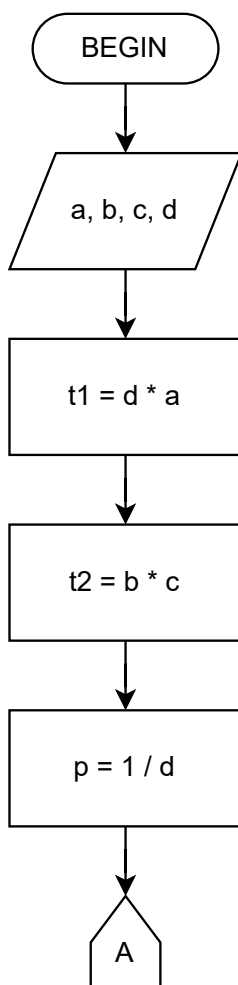
Мій порядковий номер у списку групи: 6, звідси я маю реалізувати такий варіант:

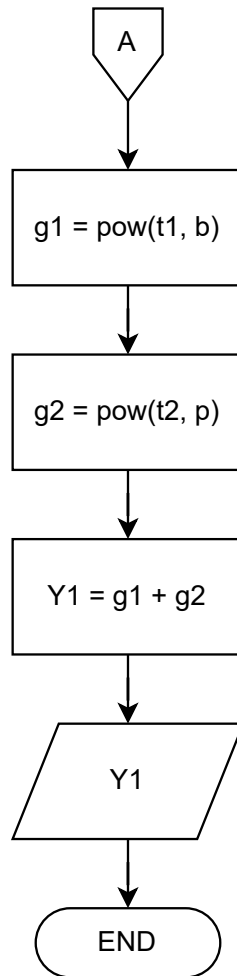
6	$Y1 = (d \cdot a)^b + (b \cdot c)^{\frac{1}{d}}$	Обчислити $y = \frac{4 \cdot r - r \cdot x}{4 \cdot x - r \cdot x}$ передбачити запобігання діленню на нуль.	Обчислити $f = \sum_{a=0}^n \left(\sum_{b=0}^p (a^b + b^a) \right)$
---	--	--	---

Рис. 1: Варіант функцій для 6 варіанту

де функцію $Y1$ я маю обчислити за допомогою лінійного алгоритму, y – розгалуженим, f – циклічним (детермінованим циклом).

Блок-схема лінійного алгоритму обчислення функції $Y1 = (d \cdot a)^b + (b \cdot c)^{\frac{1}{d}}$:





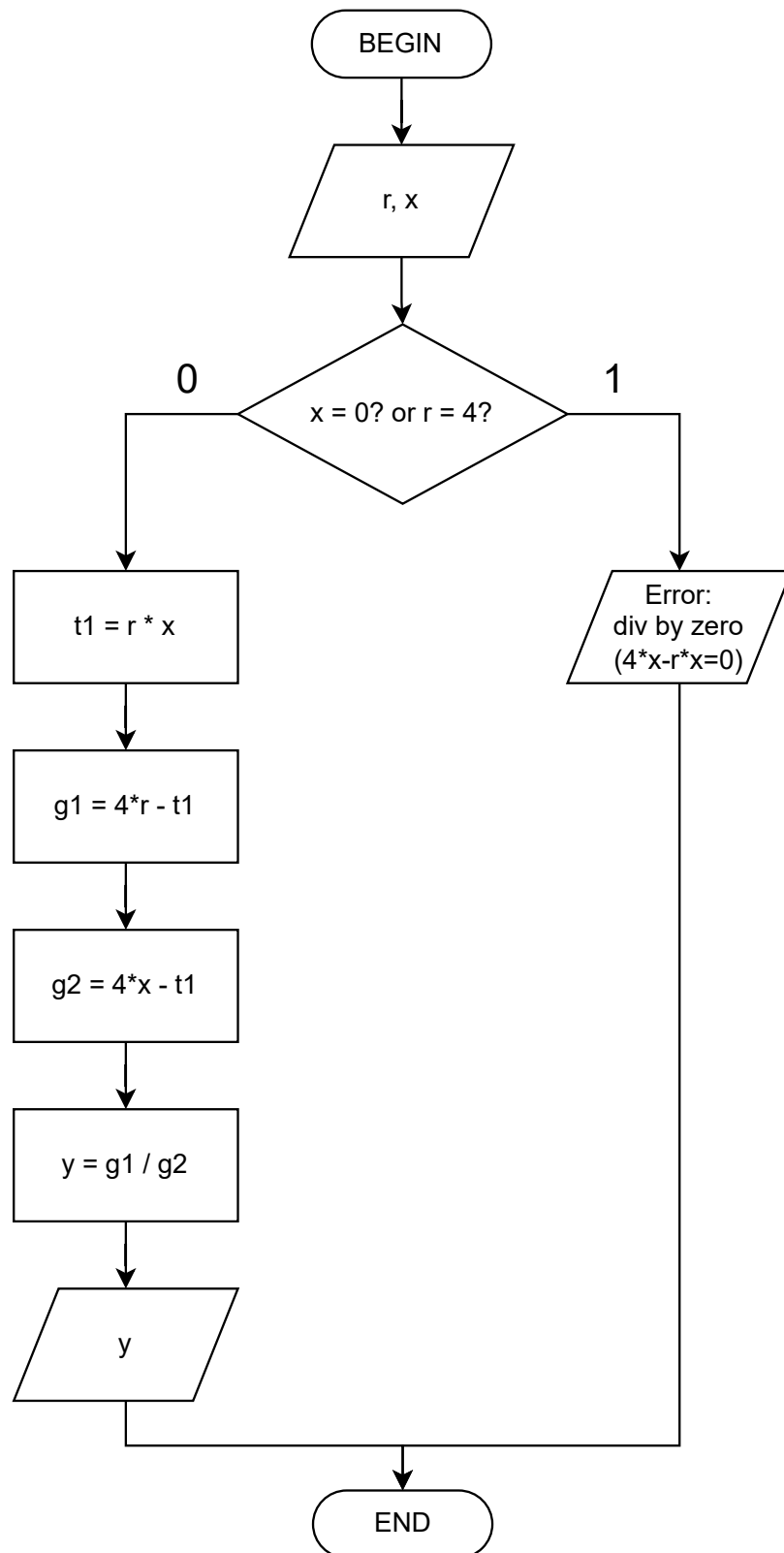
Тут функція $\text{pow}(x, y)$ – функція піднесення числа x до степеню y :

$$\text{pow}(x, y) = x^y$$

Щодо недоліків цього алгоритму: він не враховує нульове значення змінної d що може викликати помилку під час обчислення $1/d$ – запобігти помилці можна лише якщо додати умову перевірки цієї змінної що не є можливим в данному випадку, тому що при такій модифікації алгоритм перестане бути лінійним, і стане тим, що розгалужується. Тому правильність виконання алгоритму залежить від вводу користувача.

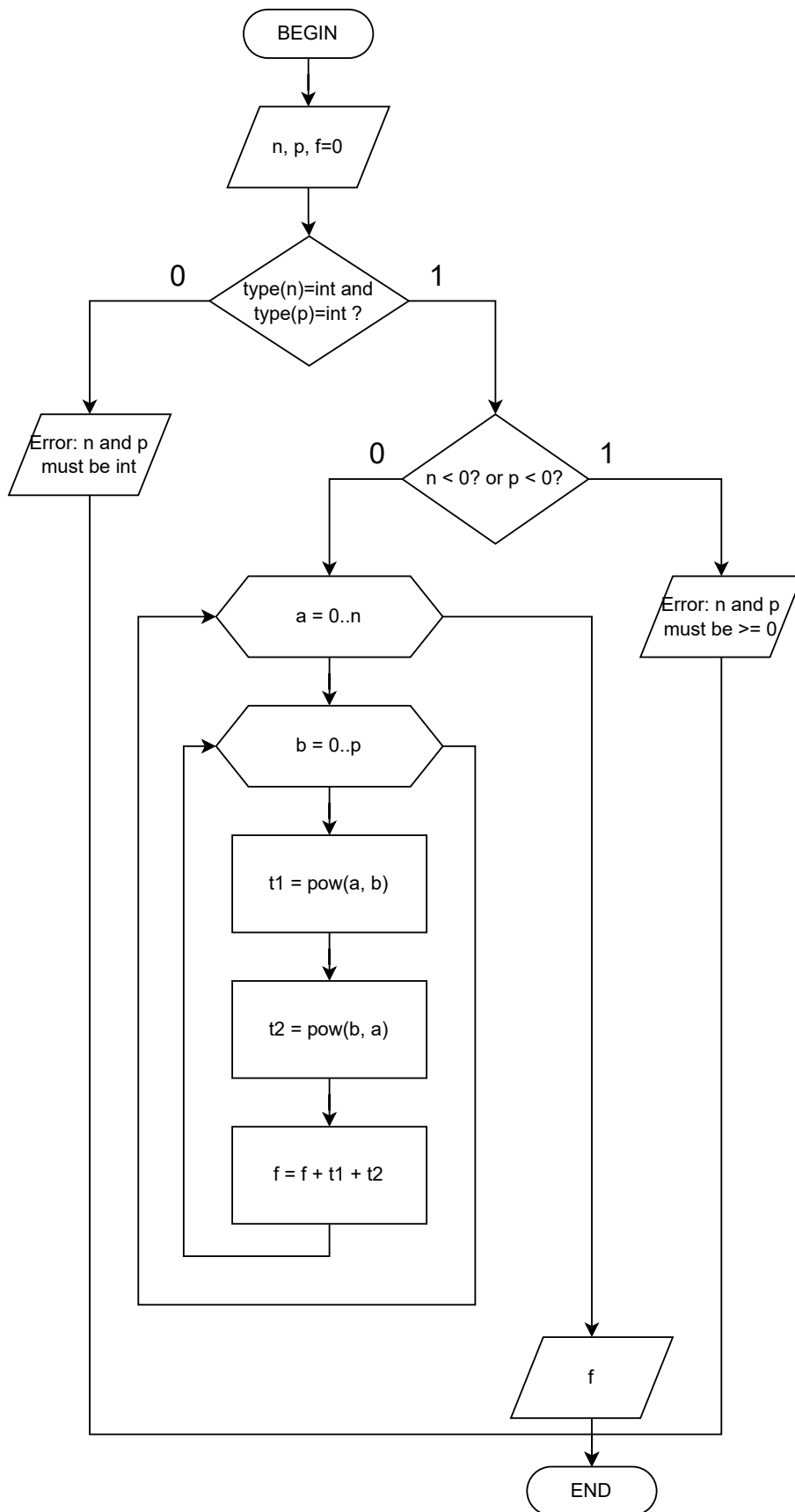
Також можна було б зазначити, якби d змінна була парною, а вираз $b \cdot c$ був від'ємним, або b парне, і $d \cdot a$ – від'ємне, і виходить при цьому виникала б помилка, коли ми хочемо взяти корінь із парним степенем від'ємного числа – але ми працюватимемо не на дійсній множині чисел, а на комплексній, яка запобігатиме таким помилкам невизначеності. Звідси також можна зробити висновок, що функція $Y1$ завжди буде належати до множини комплексних чисел (є комплексним числом).

Блок-схема розгалуженого алгоритму обчислення функції $y = \frac{4r - rx}{4x - rx}$:



Тут бачимо, що знаменник дорівнює нулю, коли $x = 0$ або $r = 4$, бо $4x - rx = x(4 - r)$ прирівнюючи яке до нуля, та розв'язавши отримаємо 2 рішення. Якщо $x = 0$ або $r = 4$, то виводиться повідомлення про помилку ділення на нуль.

Блок-схема циклічного алгоритму обчислення функції $f = \sum_{a=0}^n \left(\sum_{b=0}^p (a^b + b^a) \right)$:



Тут потрібно зазначити, що ми приймаємо $0^0 = 1$, тобто

$$\text{pow}(0, 0) = 1$$

Також зазначимо, що кількість ітерацій від початку відома (її кількість вводиться користувачем) – звідси всі цикли, які присутні в цьому алгоритмі – детерміновані. Це було видно на блок-схемі, коли для заголовку детермінованого циклу використовувався гексагон.

Для реалізації алгоритмів та GUI середовища я використовуватиму мову програмування Python і інструмент Tkinter та бібліотеку customtkinter.

Програмний код для реалізації лінійного алгоритму (src/linear_alg.py):

```

1  from __future__ import annotations
2
3  def linear_alg(*, a: int | float | complex, b: int | float | complex, c: int | float
   ↪ | complex, d: int | float | complex) -> complex:
4      """
5      Обчислює функцію  $Y1 = (d*a)^b + (b*c)^{(1/d)}$ 
6
7      a, b, c, d можуть бути int/float/complex.
8      Під час обчислень трактується як комплексні.
9      Обмеження яке в алгоритмі не враховується:  $d \neq 0$ .
10     """
11
12     t1 = d * a
13     t2 = b * c
14
15     p = 1 / d
16
17     g1 = t1 ** b
18     g2 = t2 ** p
19
20     Y1 = g1 + g2
21
22     return complex(Y1)

```

Програмний код для реалізації розгалуженого алгоритму (src/branched_alg.py):

```

1  from __future__ import annotations
2
3  def branched_alg(*, r: int | float | complex, x: int | float | complex) -> complex:
4      """
5      Обчислює функцію  $y = (4*r - r*x) / (4*x - r*x)$ 
6
7      r, x можуть бути int/float/complex.
8      Під час обчислень трактується як комплексні.
9      Обмеження яке в алгоритмі враховується:  $x \neq 0$  та  $r \neq 4$ .
10     """
11
12     if (x == 0) or (r == 4):
13
14         raise ZeroDivisionError("x != 0 and r != 4!")

```

```

15
16     t1 = r * x
17
18     g1 = 4*r - t1
19     g2 = 4*x - t1
20
21     y = g1 / g2
22
23     return complex(y)

```

Програмний код для реалізації циклічного алгоритму (src/cyclic_alg.py):

```

1  from __future__ import annotations
2
3  def cyclic_alg(*, n: int, p: int) -> int:
4
5      f: int = 0
6
7      if (
8          isinstance(n, bool) or not isinstance(n, int)    # Число n строго є типом
9          ↪ int (і не bool зокрема)
10         or isinstance(p, bool) or not isinstance(p, int) # Число p строго є типом
11         ↪ int (і не bool зокрема)
12     ):
13
14         raise TypeError("n and p must be int (not float/complex/bool)")
15
16     if (n < 0) or (p < 0):
17
18         raise ValueError("n and p must be non-negative integers (>= 0)")
19
20     for a in range(n+1):
21
22         for b in range(p+1):
23
24             t1 = a**b
25             t2 = b**a
26
27             f += t1 + t2
28
29     return f

```

Програмний код основної програми (app.py):

```

1  from __future__ import annotations
2
3  import json
4  import re
5  import tkinter as tk
6  from dataclasses import dataclass
7  from pathlib import Path
8  from tkinter import filedialog
9  from typing import Callable

```

```

10
11 import customtkinter as ctk
12
13 from src.branched_alg import branched_alg
14 from src.cyclic_alg import cyclic_alg
15 from src.linear_alg import linear_alg
16
17 MAIN_THEME: dict[str, str] = {
18     "bg": "#000000",
19     "topbar": "#000000",
20     "panel": "#090909",
21     "surface": "#111111",
22     "surface_soft": "#1B1B1B",
23     "text": "#F5F7FA",
24     "muted": "#97A1AD",
25     "accent": "#29A8FF",
26     "accent_hover": "#44B4FF",
27     "accent_text": "#06121B",
28     "segment": "#131313",
29     "segment_hover": "#1E1E1E",
30     "segment_active": "#1F2D3C",
31     "error": "#FF5B71",
32 }
33
34 INT_PATTERN = re.compile(r"^[+-]?\\d+$")
35 EPSILON = 1e-12
36
37 @dataclass(frozen=True)
38 class PageConfig:
39     page_id: str
40     nav_title: str
41     title: str
42     subtitle: str
43     fields: tuple[str, ...]
44     parser: Callable[[dict[str, str]], dict[str, object]]
45     compute: Callable[..., object]
46
47
48 def parse_complex_fields(raw_values: dict[str, str]) -> dict[str, complex]:
49     parsed: dict[str, complex] = {}
50     for key, raw in raw_values.items():
51         value = raw.strip()
52         if not value:
53             raise ValueError(f"'{key}' is required")
54
55         try:
56             parsed[key] = complex(value)
57         except ValueError as exc:
58             raise ValueError(
59                 f"'{key}' must be a complex-compatible string (examples: '1', '2.5',
60                 ↪ '1+2j')"
61             ) from exc
62     return parsed

```



```

63
64 def parse_cyclic_fields(raw_values: dict[str, str]) -> dict[str, int]:
65     parsed: dict[str, int] = {}
66     for key in ("n", "p"):
67         raw = raw_values[key].strip()
68         if not raw:
69             raise ValueError(f"'{key}' is required")
70         if not INT_PATTERN.fullmatch(raw):
71             raise TypeError(f"'{key}' must be an integer string (for example: '0',
72                 ↪ '5', '12')")
73
74         value = int(raw)
75         if value < 0:
76             raise ValueError(f"'{key}' must be >= 0")
77         parsed[key] = value
78     return parsed
79
80 def _normalize_zero(value: float) -> float:
81     return 0.0 if abs(value) < EPSILON else value
82
83
84 def _format_real(value: float) -> str:
85     normalized = _normalize_zero(value)
86     if float(normalized).is_integer():
87         return str(int(normalized))
88     return f"{normalized:.12g}"
89
90
91 def format_output_value(value: object) -> str:
92     if isinstance(value, complex):
93         real = _normalize_zero(float(value.real))
94         imag = _normalize_zero(float(value.imag))
95
96         if imag == 0.0:
97             return _format_real(real)
98
99         sign = "+" if imag >= 0 else "-"
100         return f"{_format_real(real)} {sign} {_format_real(abs(imag))}j"
101
102     if isinstance(value, float):
103         return _format_real(value)
104
105     return str(value)
106
107
108 class AlgorithmPage(ctlk.CTkFrame):
109     def __init__(
110         self,
111         master: ctlk.CTkFrame,
112         config: PageConfig,
113         fonts: dict[str, ctlk.CTkFont],
114     ) -> None:
115         super().__init__(master, corner_radius=0)

```

```

116     self.config_data = config
117     self.fonts = fonts
118     self.entries: dict[str, ctk.CTkEntry] = {}
119     self.field_labels: list[ctk.CTkLabel] = []
120     self.result_var = tk.StringVar(value="Ready")
121     self.error_var = tk.StringVar(value="")
122
123     self.grid_columnconfigure(0, weight=1)
124
125     self._build_header()
126     self._build_form()
127     self._build_actions()
128     self._build_output()
129
130     def _build_header(self) -> None:
131         self.header_card = ctk.CTkFrame(self, corner_radius=14, border_width=1)
132         self.header_card.grid(row=0, column=0, sticky="ew", pady=(0, 14))
133         self.header_card.grid_columnconfigure(0, weight=1)
134
135         self.title_label = ctk.CTkLabel(
136             self.header_card,
137             text=self.config_data.title,
138             anchor="w",
139             font=self.fonts["title"],
140         )
141         self.title_label.grid(row=0, column=0, sticky="ew", padx=20, pady=(14, 4))
142
143         self.subtitle_label = ctk.CTkLabel(
144             self.header_card,
145             text=self.config_data.subtitle,
146             anchor="w",
147             font=self.fonts["subtitle"],
148         )
149         self.subtitle_label.grid(row=1, column=0, sticky="ew", padx=20, pady=(0,
150             ↪ 12))
151
152     def _build_form(self) -> None:
153         self.form_card = ctk.CTkFrame(self, corner_radius=14, border_width=1)
154         self.form_card.grid(row=1, column=0, sticky="ew", pady=(0, 14))
155         self.form_card.grid_columnconfigure(1, weight=1)
156
157         for row, field_name in enumerate(self.config_data.fields):
158             label = ctk.CTkLabel(
159                 self.form_card,
160                 text=f"{field_name}:",
161                 width=42,
162                 anchor="w",
163                 font=self.fonts["label"],
164             )
165             label.grid(row=row, column=0, sticky="w", padx=(20, 12), pady=9)
166             self.field_labels.append(label)
167
168             entry = ctk.CTkEntry(
169                 self.form_card,

```

```

169         height=38,
170         corner_radius=10,
171         font=self.fonts["body"],
172     )
173     entry.grid(row=row, column=1, sticky="ew", padx=(0, 20), pady=9)
174     self.entries[field_name] = entry
175
176 def _build_actions(self) -> None:
177     self.actions = ctk.CTkFrame(self, fg_color="transparent")
178     self.actions.grid(row=2, column=0, sticky="ew", pady=(0, 10))
179     self.actions.grid_columnconfigure((0, 1), weight=1)
180
181     self.compute_button = ctk.CTkButton(
182         self.actions,
183         text="Compute",
184         height=42,
185         corner_radius=11,
186         font=self.fonts["button"],
187         command=self.compute,
188     )
189     self.compute_button.grid(row=0, column=0, sticky="ew", padx=(0, 7))
190
191     self.load_button = ctk.CTkButton(
192         self.actions,
193         text="Load JSON",
194         height=42,
195         corner_radius=11,
196         font=self.fonts["button"],
197         command=self.load_json,
198     )
199     self.load_button.grid(row=0, column=1, sticky="ew", padx=(7, 0))
200
201 def _build_output(self) -> None:
202     self.output_card = ctk.CTkFrame(self, corner_radius=14, border_width=1)
203     self.output_card.grid(row=3, column=0, sticky="ew")
204     self.output_card.grid_columnconfigure(0, weight=1)
205
206     self.result_title = ctk.CTkLabel(
207         self.output_card,
208         text="Result",
209         anchor="w",
210         font=self.fonts["label"],
211     )
212     self.result_title.grid(row=0, column=0, sticky="ew", padx=20, pady=(14, 4))
213
214     self.result_label = ctk.CTkLabel(
215         self.output_card,
216         textvariable=self.result_var,
217         anchor="w",
218         font=self.fonts["result"],
219     )
220     self.result_label.grid(row=1, column=0, sticky="ew", padx=20, pady=(0, 6))
221
222     self.error_label = ctk.CTkLabel(

```

```

223         self.output_card,
224         textvariable=self.error_var,
225         anchor="w",
226         justify="left",
227         wraplength=860,
228         font=self.fonts["body"],
229     )
230     self.error_label.grid(row=2, column=0, sticky="ew", padx=20, pady=(0, 14))
231
232 def apply_theme(self, palette: dict[str, str]) -> None:
233     self.configure(fg_color=palette["bg"])
234
235     for card in (self.header_card, self.form_card, self.output_card):
236         card.configure(
237             fg_color=palette["panel"],
238             border_color=palette["surface_soft"],
239         )
240
241     self.title_label.configure(text_color=palette["text"])
242     self.subtitle_label.configure(text_color=palette["muted"])
243
244     for label in self.field_labels:
245         label.configure(text_color=palette["muted"])
246
247     for entry in self.entries.values():
248         entry.configure(
249             fg_color=palette["surface"],
250             border_color=palette["surface_soft"],
251             text_color=palette["text"],
252             placeholder_text_color=palette["muted"],
253         )
254
255     self.compute_button.configure(
256         fg_color=palette["accent"],
257         hover_color=palette["accent_hover"],
258         text_color=palette["accent_text"],
259     )
260     self.load_button.configure(
261         fg_color=palette["segment"],
262         hover_color=palette["segment_hover"],
263         text_color=palette["text"],
264     )
265
266     self.result_title.configure(text_color=palette["muted"])
267     self.result_label.configure(text_color=palette["text"])
268     self.error_label.configure(text_color=palette["error"])
269
270 def _collect_raw_values(self) -> dict[str, str]:
271     return {key: entry.get() for key, entry in self.entries.items()}
272
273 def _show_result(self, value: object) -> None:
274     self.result_var.set(format_output_value(value))
275     self.error_var.set("")
276

```

```

277     def _show_error(self, error: Exception | str) -> None:
278         self.result_var.set("No result")
279         self.error_var.set(f"Error: {error}")
280
281     def _set_entries_from_dict(self, values: dict[str, str]) -> None:
282         for key, entry in self.entries.items():
283             entry.delete(0, tk.END)
284             entry.insert(0, values[key])
285
286     def load_json(self) -> None:
287         path = filedialog.askopenfilename(
288             title="Select JSON file",
289             filetypes=[("JSON files", "*.json"), ("All files", "*.*")],
290         )
291         if not path:
292             return
293
294         try:
295             raw_data = Path(path).read_text(encoding="utf-8")
296             loaded = json.loads(raw_data)
297
298             if not isinstance(loaded, dict):
299                 raise ValueError("JSON root must be an object")
300
301             expected_keys = set(self.config_data.fields)
302             actual_keys = set(loaded.keys())
303             if actual_keys != expected_keys:
304                 raise ValueError(f"Expected keys exactly: {sorted(expected_keys)}")
305
306             for key in self.config_data.fields:
307                 if not isinstance(loaded[key], str):
308                     raise TypeError(f"JSON value for '{key}' must be a string")
309
310             prepared = {key: loaded[key] for key in self.config_data.fields}
311             self._set_entries_from_dict(prepared)
312             self.error_var.set("")
313             self.result_var.set("JSON loaded. Press Compute.")
314         except Exception as exc:
315             self._show_error(exc)
316
317     def compute(self) -> None:
318         try:
319             raw_values = self._collect_raw_values()
320             parsed = self.config_data.parser(raw_values)
321             result = self.config_data.compute(**parsed)
322             self._show_result(result)
323         except Exception as exc:
324             self._show_error(exc)
325
326
327 class AlgorithmsApp(ctk.CTk):
328     def __init__(self) -> None:
329         super().__init__()
330         ctk.set_appearance_mode("dark")

```

```

331 self.title("AlgorithmsDemoApp")
332 self.resizable(False, False)
333 self.geometry("100x500")
334 self.minsize(780, 680)
335
336
337 self.current_page = "linear_alg"
338 self.nav_buttons: dict[str, ctk.CTkButton] = {}
339 self.pages: dict[str, AlgorithmPage] = {}
340
341 self.fonts: dict[str, ctk.CTkFont] = {
342     "title": ctk.CTkFont(family="Segoe UI", size=27, weight="bold"),
343     "subtitle": ctk.CTkFont(family="Segoe UI", size=13),
344     "label": ctk.CTkFont(family="Segoe UI", size=13, weight="bold"),
345     "body": ctk.CTkFont(family="Segoe UI", size=12),
346     "button": ctk.CTkFont(family="Segoe UI", size=12, weight="bold"),
347     "result": ctk.CTkFont(family="Consolas", size=18, weight="bold"),
348     "nav": ctk.CTkFont(family="Segoe UI", size=12, weight="bold"),
349 }
350
351 self.page_configs: tuple[PageConfig, ...] = (
352     PageConfig(
353         page_id="linear_alg",
354         nav_title="Linear",
355         title="Linear Algorithm",
356         subtitle="Y1 = (d*a)^b + (b*c)^(1/d)",
357         fields=("a", "b", "c", "d"),
358         parser=parse_complex_fields,
359         compute=linear_alg,
360     ),
361     PageConfig(
362         page_id="branched_alg",
363         nav_title="Branched",
364         title="Branched Algorithm",
365         subtitle="y = (4*r - r*x) / (4*x - r*x)",
366         fields=("r", "x"),
367         parser=parse_complex_fields,
368         compute=branched_alg,
369     ),
370     PageConfig(
371         page_id="cyclic_alg",
372         nav_title="Cyclic",
373         title="Cyclic Algorithm",
374         subtitle="f = sum(a^b + b^a), for a in [0..n], b in [0..p]",
375         fields=("n", "p"),
376         parser=parse_cyclic_fields,
377         compute=cyclic_alg,
378     ),
379 )
380
381 self.grid_columnconfigure(0, weight=1)
382 self.grid_rowconfigure(1, weight=1)
383
384 self._build_topbar()

```

```

385         self._build_pages()
386         self._apply_theme()
387         self.show_page(self.current_page)
388
389     def _build_topbar(self) -> None:
390         self.topbar = ctk.CTkFrame(self, corner_radius=0)
391         self.topbar.grid(row=0, column=0, sticky="ew")
392         self.topbar.grid_columnconfigure(0, weight=1)
393
394         self.nav_frame = ctk.CTkFrame(self.topbar, fg_color="transparent")
395         self.nav_frame.grid(row=0, column=0, sticky="w", padx=20, pady=(18, 12))
396
397         for index, page in enumerate(self.page_configs):
398             button = ctk.CTkButton(
399                 self.nav_frame,
400                 text=page.nav_title,
401                 width=110,
402                 height=36,
403                 corner_radius=10,
404                 font=self.fonts["nav"],
405                 border_width=1,
406                 command=lambda pid=page.page_id: self.show_page(pid),
407             )
408             button.grid(row=0, column=index, padx=(0, 8))
409             self.nav_buttons[page.page_id] = button
410
411     def _build_pages(self) -> None:
412         self.page_host = ctk.CTkFrame(self, corner_radius=0)
413         self.page_host.grid(row=1, column=0, sticky="nsew", padx=20, pady=(0, 20))
414         self.page_host.grid_rowconfigure(0, weight=1)
415         self.page_host.grid_columnconfigure(0, weight=1)
416
417         for config in self.page_configs:
418             page = AlgorithmPage(self.page_host, config, self.fonts)
419             page.grid(row=0, column=0, sticky="nsew")
420             self.pages[config.page_id] = page
421
422     def show_page(self, page_id: str) -> None:
423         self.current_page = page_id
424         self.pages[page_id].tkraise()
425         self._refresh_nav_buttons()
426
427     def _refresh_nav_buttons(self) -> None:
428         palette = MAIN_THEME
429         for page_id, button in self.nav_buttons.items():
430             if page_id == self.current_page:
431                 button.configure(
432                     fg_color=palette["segment_active"],
433                     hover_color=palette["segment_active"],
434                     text_color=palette["text"],
435                     border_color=palette["segment_active"],
436                 )
437             else:
438                 button.configure(

```

```

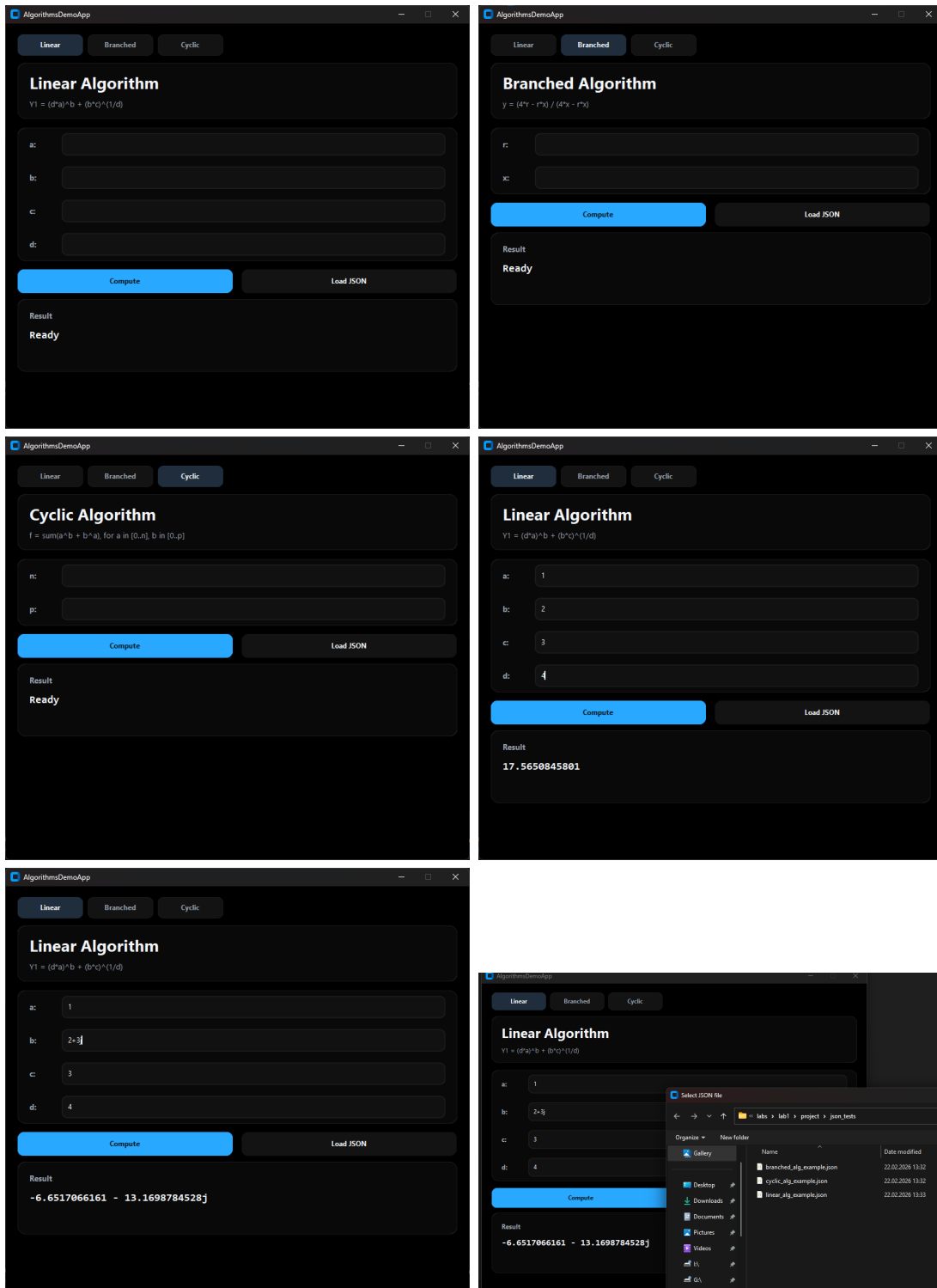
439         fg_color=palette["segment"],
440         hover_color=palette["segment_hover"],
441         text_color=palette["muted"],
442         border_color=palette["surface_soft"],
443     )
444
445     def _apply_theme(self) -> None:
446         palette = MAIN_THEME
447
448         self.configure(fg_color=palette["bg"])
449         self.topbar.configure(fg_color=palette["topbar"])
450         self.page_host.configure(fg_color=palette["bg"])
451
452         for page in self.pages.values():
453             page.apply_theme(palette)
454
455         self._refresh_nav_buttons()
456
457     def main() -> None:
458         app = AlgorithmsApp()
459         app.mainloop()
460
461     if __name__ == "__main__":
462         main()

```

Короткий опис функціоналу основної програми:

- GUI на `customtkinter` у мінімалістичному стилі (dark).
- 3 сторінки алгоритмів: `linear_alg`, `branched_alg`, `cyclic_alg`.
- Для кожної сторінки: поля вводу, кнопки `Compute` і `Load JSON`, блоки результату та помилки.
- JSON завантажується з файлу, ключі перевіряються строго під поточний алгоритм, значення мають бути рядками.
- Парсинг: для `linear/branched` – `complex(...)`, для `cyclic` – лише `int` і умови $n, p \geq 0$.
- Помилки не зупиняють програму: показуються як `Error: ....`
- Формат виводу: $1+0j \rightarrow 1$, $2.5+0j \rightarrow 2.5$, інакше $n \pm mj$.

Скріншоти демонстрації програми:



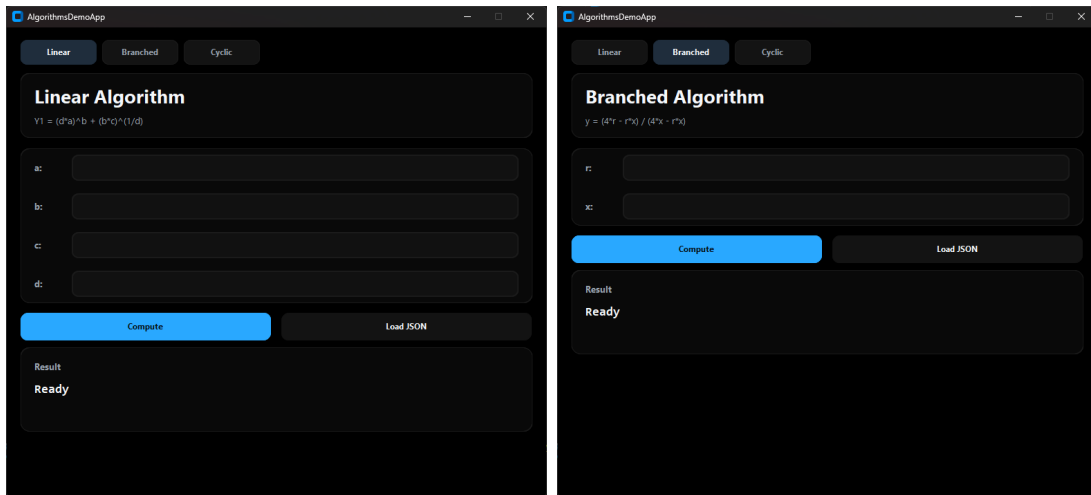


Рис. 3: Скріншоти демонстрації програми

Аналіз результатів

Реалізований лінійний алгоритм обчислює задану функцію $Y1$, за винятком випадку $d = 0$, коли на етапі обчислення $p = 1/d$ виникає ділення на нуль. Інших обмежень на область визначення (зокрема, пов'язаних із піднесенням до степеня) не накладається, оскільки обчислення виконуються в множині комплексних чисел $\mathbb{C}$, де операції піднесення до степеня визначені (за стандартними домовленостями реалізації комплексної степеневі функції).

Перевірка на нульове значення змінної d реалізована в основній програмі GUI, тоді як у самому лінійному алгоритмі вона відсутня. Це узгоджується з вимогами до лінійних алгоритмів (відсутність розгалужень): коректність забезпечується через попередню валідацію вхідних даних.

Алгоритм уніфіковано за типом результату: він завжди повертає значення типу `complex`. Такий підхід спрощує подальшу обробку результату в GUI та робить інтерфейс однорідним. Основна програма при цьому здатна коректно інтерпретувати випадки, коли результат є дійсним (тобто має нульову уявну частину), і відрізняти їх від власне комплексних значень з ненульовою уявною частиною.

Щодо часової складності лінійного алгоритму: кількість виконуваних кроків є сталою. Якщо припустити, що операція піднесення до степеня (функція `pow`) виконується за сталий час $O(1)$ для використаних типів даних, то загальна часова складність алгоритму становить

$$T = \Theta(1) \quad (\text{тобто } T \in O(1) \text{ і } T \in \Omega(1)).$$

Отже, реалізований лінійний алгоритм є коректним за умови $d \neq 0$.

Реалізований розгалужений алгоритм обчислює задану функцію y , за винятком випадків, коли знаменник дорівнює нулю. У даній реалізації ця умова перевіряється безпосередньо в алгоритмі, що унеможливорює некоректне введення та запобігає діленню на нуль. З аналізу виразу випливає, що нульовий знаменник виникає при $x = 0$ або $r = 4$, тому умови коректності мають вигляд $x \neq 0$ та $r \neq 4$.

Як і для лінійного алгоритму, результат розгалуженого алгоритму завжди повертається у вигляді комплексного числа (`complex`). Це уніфікує формат вихідних даних і дає змогу основній програмі однаково обробляти як дійсні, так і комплексні значення, зберігаючи можливість розпізнавати випадок нульової уявної частини.

Щодо часової складності: алгоритм виконує сталу кількість операцій (одну перевірку умови та фіксований набір арифметичних обчислень) і не містить циклів. За класичного

припущення про сталу вартість арифметичних операцій часова складність становить

$$T = \Theta(1) \quad (\text{тобто } T \in O(1) \text{ і } T \in \Omega(1)).$$

Таким чином, за виконання умов $x \neq 0$ та $r \neq 4$ розгалужений алгоритм працює коректно та забезпечує результативне обчислення заданої функції.

Реалізований циклічний алгоритм призначений для обчислення значення функції

$$f = \sum_{a=0}^n \left(\sum_{b=0}^p (a^b + b^a) \right).$$

Оскільки вказаний вираз містить подвійне сумування, реалізація природно виконується за допомогою двох вкладених детермінованих циклів: зовнішній цикл перебирає a від 0 до n , а внутрішній — b від 0 до p , після чого значення $a^b + b^a$ додається до акумулятора f .

Для забезпечення коректності введення у даній роботі накладено обмеження на параметри n та p : вони повинні бути цілими невід'ємними числами. У програмній реалізації це перевіряється явно: значення мають належати типу `int` (строго, без `float` та `complex`) та задовольняти $n \geq 0$, $p \geq 0$. У випадку порушення цих вимог алгоритм не виконує обчислення і формує повідомлення про помилку введених даних, що унеможлиблює некоректну інтерпретацію меж циклів.

Окремо зазначимо, що в сумі присутній доданок, який формально містить вираз 0^0 (для $a = 0$ та $b = 0$). У рамках даної реалізації прийнято стандартне для дискретних задач означення $0^0 = 1$, що також узгоджується з поведінкою мови Python (`0**0 == 1`). Це дозволяє однозначно обчислювати відповідний доданок та забезпечує коректність роботи алгоритму для всіх $n \geq 0$, $p \geq 0$.

Щодо часової складності: кількість ітерацій зовнішнього циклу дорівнює $n + 1$, внутрішнього — $p + 1$, отже загальна кількість виконань тіла циклу становить $(n + 1)(p + 1)$. Якщо вартість піднесення до степеня розглядати як сталу (за умови фіксованого розміру чисел), то часову складність циклічного алгоритму можна оцінити як

$$T(n, p) = \Theta((n + 1)(p + 1)) = \Theta(np).$$

Зауважимо, що для великих значень a та b операції a^b і b^a можуть суттєво впливати на практичний час виконання через зростання розміру чисел, однак у межах класичної оцінки за кількістю ітерацій алгоритм має квадратичну (добуткову) залежність від меж сумування.

Таким чином, при виконанні умов $n \in \mathbb{Z}$, $p \in \mathbb{Z}$, $n \geq 0$, $p \geq 0$ циклічний алгоритм працює коректно та забезпечує обчислення заданої подвійної суми.